

计算机体系结构期末复习宝典

整理依据：当前文件夹内第 0-8 章课件，以及三次作业 PDF 的课程范围。作业 PDF 抽取文本为空，疑似扫描/图片型，因此例题按课件中的核心公式、典型课堂例题和常见考试题型重构。

0. 考前抓大放小

最容易考计算题的部分：

1. 性能评价：CPU time、CPI、MIPS、Amdahl 定律。
2. 指令系统：栈/累加器/寄存器-存储器/Load-Store 结构的代码序列比较，寻址方式，MIPS 指令格式。
3. 流水线：加速比、吞吐率、效率、结构/数据/控制冲突，RAW/WAR/WAW，旁路与停顿。
4. 指令级并行：静态调度、循环展开、记分牌、Tomasulo、寄存器重命名。
5. Cache：地址划分、映像方式、失效率、AMAT、多级 Cache、3C 失效。
6. I/O 与可靠性：Amdahl 在 I/O 中的应用，MTTF/MTTR/Availability，RAID 级别。

最容易考概念辨析的部分：

1. 体系结构、组成、实现三者区别。
2. RISC/CISC，Load-Store 结构优缺点。
3. Flynn 分类：SISD、SIMD、MISD、MIMD。
4. Cache 写策略：写直达/写回，写分配/不写分配。
5. RAID 0/1/3/4/5/6/10 的性能、容量和容错。

1. 计算机系统结构基础与性能评价

1.1 必背知识点

计算机系统结构是软硬件之间的接口，传统上常指 ISA，即机器语言程序员可见的属性，包括寄存器、寻址方式、指令格式、操作类型、异常和中断等。

体系结构、组成、实现的区别：

层次 关注点 例子
--- --- ---
体系结构 Architecture 程序员可见功能 是否有乘法指令
组成 Organization 如何用逻辑结构实现 ISA 用专用乘法器还是加法器迭代实现乘法
实现 Implementation 物理实现 器件、工艺、封装、时钟、电源、散热

性能基本公式：

$$\text{CPU time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$
$$\text{CPU time} = \text{Instruction Count} \times \text{CPI} / \text{Clock Rate}$$
$$\text{CPI} = \sum (\text{指令比例}_i \times \text{CPI}_i)$$
$$\text{Speedup} = \text{原执行时间} / \text{新执行时间}$$

Amdahl 定律：

$$\text{Speedup} = 1 / ((1 - f) + f / S)$$

其中 f 是可改进部分在原执行时间中的比例， S 是该部分加速倍数。极限加速比为 $1 / (1 - f)$ 。

Flynn 分类:

类型	含义	例子
--- --- ---		
SISD	单指令流单数据流	传统单核标量机
SIMD	单指令流多数据流	向量机、GPU 的部分执行模型
MISD	多指令流单数据流	很少见
MIMD	多指令流多数据流	多核、多处理器

1.2 例题：Amdahl 定律

题目：某程序中浮点运算占原执行时间的 40%。若把浮点部件加速 20 倍，程序总加速比是多少？如果浮点部件无限快，最大加速比是多少？

讲解：

$$f = 0.4, S = 20 \quad \text{Speedup} = 1 / ((1 - 0.4) + 0.4 / 20) = 1 / (0.6 + 0.02) = 1 / 0.62 = 1.61$$

若浮点部件无限快， $S \rightarrow \infty$ ：

$$\text{Speedup}_{\max} = 1 / (1 - 0.4) = 1 / 0.6 = 1.667$$

结论：局部优化会受到未优化部分限制。考试中看到“某部分占比 + 加速倍数”，优先套 Amdahl。

1.3 例题：CPU 性能比较

题目：机器 A 的时钟周期为 1 ns，CPI 为 2.0；机器 B 的时钟周期为 1.5 ns，CPI 为 1.2。两者执行同一程序，指令条数相同，谁更快？

讲解：

只比较 $\text{CPI} \times \text{Clock Cycle Time}$ ：

$$A: 2.0 \times 1 \text{ ns} = 2.0 \text{ ns/指令} \quad B: 1.2 \times 1.5 \text{ ns} = 1.8 \text{ ns/指令}$$

B 更快，加速比：

$$\text{Speedup}_{B \text{ over } A} = 2.0 / 1.8 = 1.11$$

注意：主频高不一定快，必须同时看 CPI 和指令条数。

2. 指令系统结构 ISA

2.1 必背知识点

指令系统常见操作数组织方式：

结构	特点	$Z = X + Y$ 示例
--- --- ---		

| 栈结构 | 操作数隐含在栈顶 | push X; push Y; add; pop Z |

| 累加器结构 | 一个隐含累加器 | load X; add Y; store Z |

| 寄存器-存储器 RM | ALU 可直接访问存储器 | load R1, X; add R1, Y; store R1, Z |

| Load-Store/RR | ALU 只操作寄存器 | load R1, X; load R2, Y; add R3, R1, R2; store R3, Z |

RISC 常见特征:

1. 指令格式规整，长度固定。
2. Load-Store 结构，访存指令与 ALU 指令分离。
3. 大量通用寄存器。
4. 寻址方式较少。
5. 便于流水线实现。

MIPS 常见格式:

| 格式 | 用途 | 关键字段 |

|---|---|---|

| R 型 | 寄存器-寄存器运算 | op, rs, rt, rd, shamt, funct |

| I 型 | 立即数、访存、分支 | op, rs, rt, immediate |

| J 型 | 跳转 | op, target |

常见寻址方式:

1. 立即寻址: 操作数在指令中。
2. 寄存器寻址: 操作数在寄存器中。
3. 基址/偏移寻址: Mem[Register + offset], MIPS 访存常用。
4. PC 相对寻址: 分支目标相对 PC。
5. 伪直接寻址: 跳转指令常见。

2.2 例题: 不同 ISA 写代码

题目: 设 X, Y, Z 均在存储器中, 写出 $Z = X + Y$ 在栈、累加器、寄存器-存储器、Load-Store 四类机器上的代码。

讲解:

```
# 栈结构 push X push Y add pop Z
# 累加器结构 load X add Y store Z
# 寄存器-存储器结构 load R1, X add R1, Y store R1, Z
# Load-Store 结构 load R1, X load R2, Y add R3, R1, R2 store R3, Z
```

答题关键: Load-Store 结构的 ALU 指令不能直接用内存操作数, 所以必须先 load 到寄存器。

2.3 例题: 平均 CPI

题目: 某程序指令构成为: ALU 50%, Load 25%, Store 10%, Branch 15%。各类 CPI 分别为 1、2、2、3。求平均 CPI。

讲解:

$$CPI = 0.50 \times 1 + 0.25 \times 2 + 0.10 \times 2 + 0.15 \times 3 = 0.50 + 0.50 + 0.20 + 0.45 = 1.65$$

考试中比例必须先化为小数, 最后单位是“周期/指令”。

3. 流水线

3.1 必背知识点

流水线的目标是提高吞吐率，而不是降低单条指令的延迟。理想 k 级流水线执行 n 条指令：

非流水时间 = $n \times k \times \Delta t$ □ 流水时间 = $(k + n - 1) \times \Delta t$ □ 加速比 = $n \times k / (k + n - 1)$ □ n 很大时，加速比趋近 k □ 吞吐率 = $n / ((k+n-1) \Delta t)$ □ 效率 = 加速比 / k

流水线冲突：

| 冲突 | 原因 | 解决 |

|---|---|---|

| 结构冲突 | 资源不够 | 增加资源、分时、停顿 |

| 数据冲突 RAW | 后指令读前指令结果 | 旁路/转发、停顿、调度 |

| 数据冲突 WAR | 后指令先写，破坏前指令读 | 寄存器重命名、顺序读 |

| 数据冲突 WAW | 后指令先写，破坏最终写序 | 寄存器重命名、顺序提交 |

| 控制冲突 | 分支改变 PC | 预测、延迟槽、提前判断 |

五级 MIPS 流水线：

IF 取指 → ID 译码/读寄存器 → EX 执行/地址计算 → MEM 访存 → WB 写回

3.2 例题：流水线加速比

题目：一条指令分为 5 个阶段，每阶段 1 ns。执行 100 条指令，理想流水线相对非流水线的加速比是多少？

讲解：

非流水时间 = $100 \times 5 \times 1 = 500$ ns □ 流水时间 = $(5 + 100 - 1) \times 1 = 104$ ns □ 加速比 = $500 / 104 = 4.81$

如果指令数趋于无穷，加速比才接近 5。不要直接写 5。

3.3 例题：数据相关判断

题目：判断以下指令间的数据相关类型。

I1: ADD R1, R2, R3 □ I2: SUB R4, R1, R5 □ I3: OR R1, R6, R7 □ I4: AND R8, R1, R9

讲解：

1. I1 → I2: I1 写 R1, I2 读 R1, 是 RAW 真相关。
2. I2 → I3: 无同一寄存器读写相关。
3. I1 → I3: 两者都写 R1, 是 WAW 输出相关。
4. I2 → I3: I2 读 R1, I3 写 R1, 是 WAR 反相关。
5. I3 → I4: I3 写 R1, I4 读 R1, 是 RAW。

解题口诀：先看同一个寄存器，再看“前后指令”的读写顺序：写后读 RAW，读后写 WAR，写后写 WAW。

3.4 例题：分支惩罚下的 CPI

题目：理想流水线 CPI 为 1，分支指令占 20%，分支预测失败率 10%，预测失败惩罚 3 个周期。求实际 CPI。

讲解：

$$\text{额外 CPI} = \text{分支比例} \times \text{失败率} \times \text{惩罚} = 0.20 \times 0.10 \times 3 = 0.06 \quad \text{实际 CPI} = 1 + 0.06 = 1.06$$

控制冲突题目通常就是“发生频率 × 代价”。

4. 向量处理机

4.1 必背知识点

向量处理适合对数组、大规模数据做相同操作，核心是开发数据级并行。

常见结构：

1. 存储器-存储器型向量机：向量操作数直接来自存储器，结果也回存储器。
2. 寄存器-寄存器型向量机：向量先装入向量寄存器，运算后再写回存储器，典型如 CRAY-1。

向量执行常见概念：

1. 向量长度 VL：一次向量指令处理的元素个数。
2. 启动时间 startup time：流水线装满前的开销。
3. 链接 chaining：前一功能部件产生的元素可直接送入后一功能部件，减少等待。
4. 分段开采 strip mining：当数组长度大于最大向量长度时，分多段处理。

4.2 例题：分段开采

题目：某向量寄存器最大长度 MVL = 64，要处理长度 N = 200 的数组。需要几段？每段 VL 是多少？

讲解：

$$200 = 64 + 64 + 64 + 8$$

需要 4 段，前三段 VL = 64，最后一段 VL = 8。

答题时说明：循环每次处理 min(MVL, 剩余元素数)。

4.3 例题：向量机为什么快

题目：为什么向量处理机比标量循环更适合 $A[i] = B[i] + C[i]$ ？

讲解：

标量循环每次迭代都需要取指、译码、分支判断和循环控制。向量指令把多个元素的相同操作合成一条指令，减少指令条数和分支开销，并可让功能部件流水化。若有多个存储体，还可连续供数，提高吞吐率。

5. 指令级并行、动态调度与 Tomasulo

5.1 必背知识点

指令级并行 ILP：

$$\text{ILP} = \text{指令数} / \text{所需周期数} \quad \text{IPC} = \text{Instructions Per Cycle} \quad \text{CPI} = 1 / \text{IPC}$$

流水线实际 CPI：

$CPI = \text{理想 CPI} + \text{结构冲突停顿} + \text{数据冲突停顿} + \text{控制冲突停顿}$

静态调度由编译器完成，常见方法：

1. 指令重排：把无关指令插入相关指令之间。
2. 循环展开：减少分支开销，暴露更多独立指令。
3. 寄存器重命名：消除 WAR/WAW 名相关。

动态调度由硬件完成：

| 方法 | 核心思想 | 解决问题 |

| --- | --- | --- |

| 记分牌 Scoreboard | 集中记录功能部件和寄存器状态 | 允许乱序执行，但遇 WAR/WAW 可能停顿 |

| Tomasulo | 保留站 + 公共数据总线 CDB + 寄存器重命名 | 消除 WAR/WAW，减少 RAW 等待 |

| ROB | 重排序缓冲 | 支持精确异常和按程序顺序提交 |

Tomasulo 三个阶段：

1. Issue：若保留站空闲，发射；源操作数可用则读值，否则记录生产者 tag。
2. Execute：源操作数就绪后执行。
3. Write result：通过 CDB 广播结果，等待者按 tag 捕获。

5.2 例题：循环调度

题目：有如下循环，假设 L.D → ADD.D 需要 1 个停顿，ADD.D → S.D 需要 2 个停顿，分支也有 1 个停顿。对代码做简单调度。

```
Loop: □ L.D    F0, 0(R1) □ ADD.D  F4, F0, F2 □ S.D    F4, 0(R1) □ DADDIU R1, R1, #-8 □ BNE    R1, R2, Loop
```

讲解：

可把与浮点数据无关的地址更新和分支提前，填补部分空隙：

```
Loop: □ L.D    F0, 0(R1) □ DADDIU R1, R1, #-8 □ ADD.D  F4, F0, F2 □ BNE    R1, R2, Loop □ S.D    F4, 8(R1)
```

为什么 S.D 地址变成 8(R1)：因为 R1 已经提前减了 8，要存回原来的地址，必须用 8(R1) 修正。

5.3 例题：循环展开

题目：把上题循环展开 4 次时，为什么要使用 F0/F4, F6/F8, F10/F12, F14/F16 等不同寄存器？

讲解：

如果四次展开都使用同一组寄存器，会产生大量 WAW 和 WAR 名相关，限制重排。使用不同寄存器相当于手工寄存器重命名，可以让四次迭代的 Load、Add、Store 互相独立，编译器能把多个 Load 放在前面，再集中执行 Add 和 Store，从而隐藏延迟。

5.4 例题：Tomasulo 消除名相关

题目：判断 Tomasulo 如何处理下面的相关：

```
I1: DIV.D F6, F0, F2 □ I2: ADD.D F8, F6, F4 □ I3: SUB.D F6, F10, F12
```

讲解：

1. I1 → I2: I2 读 I1 产生的 F6, 是 RAW 真相关, 不能消除, 必须等 I1 结果。
2. I1 → I3: 两者都写 F6, 是 WAW 名相关。Tomasulo 给两个写 F6 的指令分配不同保留站 tag, 寄存器结果状态只保留最新生产者, 因此可避免错误覆盖。
3. I2 → I3: I2 读 F6, I3 写 F6, 是 WAR 名相关。由于 I2 需要的是 I1 的 tag, 而不是“寄存器 F6 这个名字”, 所以 I3 改写 F6 的名字不会破坏 I2。

结论: Tomasulo 不能消除 RAW 真相关, 但能通过寄存器重命名消除 WAR/WAW。

6. Cache 与存储层次

6.1 必背知识点

存储层次建立在局部性原理上：

1. 时间局部性：最近访问过的数据/指令很可能再次访问。
2. 空间局部性：访问某地址后, 邻近地址很可能被访问。

Cache 基本问题：

问题 选项
--- ---
放哪里 直接映像、组相联、全相联
找不找得到 tag + valid 位比较
满了替换谁 随机、FIFO、LRU
写命中怎么办 写直达、写回
写不命中怎么办 写分配、不写分配

映像方式：

直接映像: $\text{cache line} = \text{memory block} \bmod \text{cache line 数}$	组相联: $\text{set} = \text{memory block} \bmod \text{set 数}$	全相联: 任意块可放任意行
---	--	---------------

地址划分：

块内偏移位 = $\log_2(\text{块大小})$	索引位 = $\log_2(\text{Cache 行数或组数})$	Tag 位 = 地址总位数 - 索引位 - 块内偏移位
------------------------------	------------------------------------	-----------------------------

平均访存时间 AMAT：

$\text{AMAT} = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$

多级 Cache：

$\text{AMAT} = \text{L1 Hit Time} + \text{L1 Miss Rate} \times (\text{L2 Hit Time} + \text{L2 Miss Rate} \times \text{Memory Penalty})$

注意：二级 Cache 的 miss rate 可能是局部失效率, 也可能是全局失效率。全局失效率 = L1 miss rate × L2 local miss rate。

3C 失效：

类型 原因 降低方法

|---|---|---|

| Compulsory 强制失效 | 第一次访问 | 预取、增大块 |

| Capacity 容量失效 | Cache 装不下工作集 | 增大 Cache |

| Conflict 冲突失效 | 映像位置冲突 | 提高相联度、Victim Cache |

6.2 例题：Cache 地址划分

题目：32 位地址，Cache 容量 16 KB，块大小 32 B，直接映像。求 offset、index、tag 位数。

讲解：

块大小 = 32 B = 2^5 B，所以 offset = 5 位
Cache 行数 = 16 KB / 32 B = 16384 / 32 = 512 = 2^9 ，所以 index = 9 位
tag = 32 - 9 - 5 = 18 位

答案：tag 18 位，index 9 位，offset 5 位。

6.3 例题：组相联地址划分

题目：32 位地址，Cache 容量 64 KB，块大小 64 B，4 路组相联。求组索引位和 tag 位。

讲解：

offset = $\log_2(64) = 6$ 位
Cache 总行数 = 64 KB / 64 B = 1024 行
组数 = 1024 / 4 = 256 = 2^8
index = 8 位
tag = 32 - 8 - 6 = 18 位

注意：组相联的 index 看“组数”，不是总行数。

6.4 例题：AMAT

题目：L1 命中时间 1 cycle，失效率 5%，失效代价 50 cycles。求 AMAT。

讲解：

$AMAT = 1 + 0.05 \times 50 = 3.5 \text{ cycles}$

若题目问“每条指令平均访存 1.3 次”，则内存停顿 CPI 为：

$Memory \text{ stall CPI} = 1.3 \times 0.05 \times 50 = 3.25$

看清题目是“每次访存”还是“每条指令”。

6.5 例题：多级 Cache

题目：L1 命中时间 1 cycle，L1 失效率 4%；L2 命中时间 8 cycles，L2 局部失效率 20%；主存代价 100 cycles。求 AMAT。

讲解：

L2 平均服务时间 = $8 + 0.20 \times 100 = 28 \text{ cycles}$
 $AMAT = 1 + 0.04 \times 28 = 2.12 \text{ cycles}$

如果题目给的是 L2 全局失效率 0.8%，则公式应写成：

$AMAT = L1 \text{ hit time} + L1 \text{ miss rate} \times L2 \text{ hit time} + L2 \text{ global miss rate} \times \text{memory penalty}$

6.6 例题：直接映像冲突

题目：Cache 有 8 行，块大小 1 word，访问块序列 0, 8, 0, 8, 0, 8，直接映像 Cache 初始为空，命中率是多少？

讲解：

直接映像行号：

```
line = block number mod 8  $\square$  0 mod 8 = 0  $\square$  8 mod 8 = 0
```

块 0 和块 8 总是争同一行。访问过程全是失效：

```
0 miss, 8 miss, 0 miss, 8 miss, 0 miss, 8 miss
```

命中率 $0/6 = 0\%$ 。这是典型 conflict miss。

7. I/O、可靠性与 RAID

7.1 必背知识点

I/O 系统组成：

1. I/O 设备：磁盘、SSD、键盘、网卡等。
2. I/O 接口/控制器：连接设备与系统总线。
3. I/O 软件：驱动、中断处理、操作系统 I/O 层。
4. DMA：设备与内存直接传输，减少 CPU 参与。

I/O 性能常用指标：

1. 响应时间：单个请求从发出到完成的时间。
2. 吞吐率：单位时间完成的 I/O 请求数或传输数据量。
3. 利用率：设备忙碌时间比例。

可靠性公式：

```
MTTF = Mean Time To Failure, 平均无故障时间  $\square$  MTTR = Mean Time To Repair, 平均修复时间  $\square$  MTBF = MTTF + MTTR  $\square$  Availability = MTTF / (MTTF + MTTR)  $\square$  串联系统失效率 = 各部件失效率之和 =  $\sum (1/MTTF\_i)$   $\square$  系统 MTTF = 1 / 系统失效率
```

RAID 速记：

级别	核心	容错	读性能	写性能	容量利用率
----	----	----	-----	-----	-------

---	---	---	---	---	---
-----	-----	-----	-----	-----	-----

RAID 0	条带化	无	高	高	100%
--------	-----	---	---	---	------

RAID 1	镜像	可坏一个镜像	高	中	50%
--------	----	--------	---	---	-----

RAID 3	位/字节级条带 + 专用校验盘	可坏 1 块	顺序读好	小写差	(N-1)/N
--------	-----------------	--------	------	-----	---------

RAID 4	块级条带 + 专用校验盘	可坏 1 块	随机读好	校验盘瓶颈	(N-1)/N
--------	--------------	--------	------	-------	---------

RAID 5	块级条带 + 分布式校验	可坏 1 块	好	小写有读改写	(N-1)/N
--------	--------------	--------	---	--------	---------

RAID 6	双分布式校验	可坏 2 块	好	写代价更大	(N-2)/N
--------	--------	--------	---	-------	---------

7.2 例题：I/O 的 Amdahl 效应

题目：某系统原来 I/O 时间占总执行时间 10%。若 CPU 速度提高 10 倍，而 I/O 不变，总执行时间变为原来的多少？总体加速比是多少？

讲解：

设原总时间为 1：

$$\text{CPU 部分} = 0.90 \quad \text{I/O 部分} = 0.10 \quad \text{新时间} = 0.90 / 10 + 0.10 = 0.09 + 0.10 = 0.19 \quad \text{总加速比} = 1 / 0.19 = 5.26$$

结论：CPU 越快，I/O 越容易成为瓶颈。

7.3 例题：系统 MTTF

题目：一个系统由 10 块磁盘、1 个控制器、1 个电源、1 个风扇组成。磁盘 MTTF 均为 1,000,000 小时，控制器 MTTF 为 500,000 小时，电源和风扇 MTTF 均为 200,000 小时。任一部件失效都会导致系统失效，求系统 MTTF。

讲解：

串联系统失效率相加：

$$\lambda = 10/1,000,000 + 1/500,000 + 1/200,000 + 1/200,000 = 0.000010 + 0.000002 + 0.000005 + 0.000005 = 0.000022 \quad \text{MTTF}_{\text{system}} = 1 / 0.000022 = 45,454.5 \text{ 小时}$$

若题目还有 1 个 SCSI 线缆且 MTTF 为 1,000,000 小时，则再加 1/1,000,000，系统 MTTF 约为：

$$\lambda = 0.000023 \quad \text{MTTF} = 43,478 \text{ 小时}$$

7.4 例题：可用性

题目：某设备 MTTF = 1000 小时，MTTR = 2 小时，求 Availability。

讲解：

$$\text{Availability} = \text{MTTF} / (\text{MTTF} + \text{MTTR}) = 1000 / 1002 = 0.9980 = 99.80\%$$

可用性高不等于永不坏，也取决于修复时间。

7.5 例题：RAID 容量

题目：8 块 4 TB 磁盘分别组成 RAID 0、RAID 1、RAID 5、RAID 6、RAID 10，有效容量是多少？

讲解：

$$\begin{aligned} \text{RAID 0: } 8 \times 4 &= 32 \text{ TB} \quad \text{RAID 1: 需要镜像, 容量约 } 32 / 2 = 16 \text{ TB} \quad \text{RAID 5: 损失 1 块校验盘容量, } (8 - 1) \times 4 = 28 \text{ TB} \\ \text{RAID 6: 损失 2 块校验盘容量, } (8 - 2) \times 4 &= 24 \text{ TB} \quad \text{RAID 10: 镜像后条带, 容量约 } 32 / 2 = 16 \text{ TB} \end{aligned}$$

RAID 0 容量和性能高，但没有容错；RAID 6 容错强但写代价更高。

8. 作业专题强化

这一章按三次作业来复盘。作业题比课件概念更接近考试风格，尤其是性能评价、流水线时空图、预约表、记分牌和 Tomasulo。

8.1 第一次作业：性能评价与平均值

题目来源：教材第 1 章课后习题 1.2、1.7、1.8、1.10、1.11。

作业覆盖点：

1. 体系结构、计算机组成、计算机实现的区别。
2. 指令条数、CPI、MIPS、程序执行时间。
3. 算术平均、几何平均、调和平均的适用场景。
4. 改进某类指令 CPI 后的总 CPI 与加速比。

例题 1：CPI、MIPS 与执行时间

题目：某程序有四类指令，条数分别为 45000、8000、75000、1500，各类 CPI 分别为 1、4、2、2。处理器主频 400 MHz。求总指令条数、平均 CPI、MIPS 和执行时间。

讲解：

$IC = 45000 + 8000 + 75000 + 1500 = 129500$ 总周期数 $= 45000 \times 1 + 8000 \times 4 + 75000 \times 2 + 1500 \times 2 = 230000$ 平均
 $CPI = 230000 / 129500 = 1.776$ $MIPS = \text{主频 (MHz)} / CPI = 400 / 1.776 = 225.2$ $MIPS$ 执行时间 $= \text{总周期数} / \text{主频} = 230000 / (400 \times 10^6) = 5.75 \times 10^{-4} \text{ s} = 575 \text{ us}$

易错点：MIPS 是“每秒百万条指令”，但不能只看 MIPS 判断机器快慢，因为不同机器可能执行不同指令条数。

例题 2：三种平均值怎么选

题目：三台机器 A、B、C 在四个程序上的 MIPS 如下：

程序	A	B	C
----	---	---	---

1	100	10	5
---	-----	----	---

2	0.1	1	5
---	-----	---	---

3	0.2	0.1	2
---	-----	-----	---

4	1	0.15	1
---	---	------	---

讲解：

算术平均：

$A = (100 + 0.1 + 0.2 + 1) / 4 = 25.325$ $B = (10 + 1 + 0.1 + 0.15) / 4 = 2.8125$ $C = (5 + 5 + 2 + 1) / 4 = 3.25$

几何平均：

$A = (100 \times 0.1 \times 0.2 \times 1)^{(1/4)} = 1.189$ $B = (10 \times 1 \times 0.1 \times 0.15)^{(1/4)} = 0.622$ $C = (5 \times 5 \times 2 \times 1)^{(1/4)} = 2.659$

调和平均：

$A = 4 / (1/100 + 1/0.1 + 1/0.2 + 1/1) = 0.250$ $B = 4 / (1/10 + 1/1 + 1/0.1 + 1/0.15) = 0.225$ $C = 4 / (1/5 + 1/5 + 1/2 + 1/1) = 2.105$

考试判断：

1. 算术平均适合“同一机器运行等权程序时间”一类直接平均，但容易被极端值误导。
2. 几何平均适合比较归一化后的相对性能。
3. 调和平均适合平均“速率”，例如同一工作量下平均 MIPS。

例题 3：改进某类指令 CPI

题目：原程序中 30% 指令 CPI 为 3，70% 指令 CPI 为 1.25。若方案一把其中 40% 指令的 CPI 降为 20/64，方案二把 70% 指令 CPI 降为 1.5，哪个更好？

讲解：

原平均 CPI：

$$CPI_{old} = 0.3 \times 3 + 0.7 \times 1.25 = 1.775$$

方案一：

$$CPI_1 = 0.4 \times (20/64) + 0.6 \times 3 = 1.925$$

方案二：

$$CPI_2 = 0.3 \times 3 + 0.7 \times 1.5 = 1.95$$

若与某个原始基准 CPI 2.375 比较：

$$Speedup_1 = 2.375 / 1.925 = 1.23 \quad Speedup_2 = 2.375 / 1.95 = 1.22$$

答题关键：先写清楚“哪些指令比例被改进”，再算新 CPI，最后用 $Speedup = CPI_{old} / CPI_{new}$ 。

8.2 第二次作业：流水线与预约表

题目来源：教材第 3 章课后习题 3.3、3.6、3.7、3.10、3.12。

作业覆盖点：

1. 分支处理：预测失败、预测成功、延迟分支。
2. 非线性流水线预约表、禁止表、冲突向量。
3. 吞吐率 TP、最大吞吐率 TP_{max}、效率 E。
4. 功能部件重复设置对流水线性能的影响。
5. 分支 CPI 对总加速比的影响。

例题 1：分支处理三种办法

题目：说明预测分支失败、预测分支成功、延迟分支三种方法。

讲解：

1. 预测分支失败：默认分支不发生，继续取顺序指令；如果实际发生分支，再清空错误取出的指令，转到目标地址。
2. 预测分支成功：默认分支发生，优先从分支目标地址取指；如果实际不发生，再恢复顺序路径。
3. 延迟分支：把分支后一条或几条指令作为延迟槽，无论分支是否发生都执行，编译器尽量填入有用指令。

共同点：都在减少控制相关造成的流水线停顿。本质区别是“猜哪条路径”和“能否用编译器填槽”。

例题 2：线性流水线吞吐率与效率

题目：某 4 段流水线各段时间为 50 ns、50 ns、100 ns、200 ns，连续输入 10 个任务。求吞吐率和效率。

讲解：

流水线时钟周期由最慢段决定：

$$\Delta t = \max(50, 50, 100, 200) = 200 \text{ ns} \quad \square \quad \text{总时间} = (\text{段数} + \text{任务数} - 1) \times \Delta t = (4 + 10 - 1) \times 200 \text{ ns} = 2600 \text{ ns} \quad \square \quad \text{TP} = 10 / 2600 \text{ ns} = 1 / 260 \text{ ns}$$

效率常写成：

$$E = \text{实际忙碌时间} / \text{总可用流水段时间} \quad \square \quad = 10 \times (50 + 50 + 100 + 200) / (4 \times 2600) \quad \square \quad = 4000 / 10400 = 38.46\%$$

如果采用细分瓶颈段或重复设置功能段，总时间会下降，效率也会变化。关键先画时空图。

例题 3：非线性流水线禁止表与冲突向量

题目：某非线性流水线预约表中，同一功能段被同一任务在时刻集合 {1, 3, 6} 使用，求这部分导致的禁止延迟。

讲解：

禁止延迟来自同一行中任意两个预约时刻的差：

$$3 - 1 = 2 \quad \square \quad 6 - 3 = 3 \quad \square \quad 6 - 1 = 5$$

因此禁止表含 {2, 3, 5}。若多行预约表都给出时刻集合，则对每行都求差，再取并集。

冲突向量写法：

$$C = (c_m \dots c_2 c_1)$$

其中 $c_i = 1$ 表示延迟 i 禁止， $c_i = 0$ 表示允许。调度时每发射一个任务，就根据冲突向量决定下一次可用延迟。

例题 4：分支对 CPI 的三种情况

题目：基础流水线 CPI 为 1，20% 为条件分支，5% 为无条件跳转。条件分支中 60% 发生跳转。预测分支失败时发生分支需 2 周期惩罚，不发生无惩罚；无条件跳转惩罚 1 周期。求 CPI。

讲解：

$$\text{CPI} = 1 + \text{条件分支比例} \times \text{发生比例} \times \text{惩罚} + \text{无条件跳转比例} \times \text{惩罚} \quad \square \quad = 1 + 0.20 \times 0.60 \times 2 + 0.05 \times 1 \quad \square \quad = 1.29$$

若预测分支成功，则条件分支不发生时才付惩罚：

$$\text{CPI} = 1 + 0.20 \times 0.40 \times 2 + 0.05 \times 1 \quad \square \quad = 1.21$$

若延迟分支能填充一部分延迟槽，就把“未填满的比例 $\times$ 惩罚”作为额外 CPI。

8.3 第三次作业：记分牌与 Tomasulo

题目来源：第 3 次课后作业 PDF，题干直接给出记分牌与 Tomasulo 调度题。

作业覆盖点：

1. 记分牌算法思想。
2. Tomasulo 算法思想。
3. 给定浮点指令序列，填写“流出/读操作数/执行/写回”周期。

- 在指定周期填写功能部件状态表和寄存器结果状态表。

作业中使用的典型指令序列：

```
L.D    F6, 34(R2) □ L.D    F2, 45(R3) □ ADD.D F8, F6, F2 □ MUL.D F4, F2, F6 □ SUB.D F8, F4, F2 □ ADD.D F4, F2, F6
```

假设功能部件：整数加法、浮点加法、浮点乘法、LOAD 部件各 2 个；LOAD、浮点加法、浮点乘法执行周期分别为 1、2、10，且尚未采用流水。

例题 1：记分牌算法怎么想

记分牌核心思想：

- 集中维护功能部件状态、寄存器结果状态和操作数就绪状态。
- 若没有结构冲突和 WAW，指令可以流出。
- 若源操作数可读且不会造成 WAR，进入读操作数。
- 操作数读完后执行。
- 写回时检查是否会破坏前面未读操作数的指令，避免 WAR。

四阶段：

```
Issue / 流出 □ Read Operands / 读操作数 □ Execute / 执行 □ Write Result / 写回
```

记分牌能乱序执行，但 WAR/WAW 仍可能造成停顿。

例题 2：记分牌时间表怎么看

题目：对上述指令序列，作业中给出的记分牌调度结果为：

指令	流出	读操作数	执行完成	写回
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	2	3	4	5
ADD.D F8, F6, F2	3	6	8	9
MUL.D F4, F2, F6	4	6	16	17
SUB.D F8, F4, F2	10	18	20	21
ADD.D F4, F2, F6	18	19	21	22

讲解：

- 两条 Load 最早流出，因为 Load 部件有 2 个，且目标寄存器不同。
- ADD.D F8, F6, F2 要等 F6、F2 都写回后才能读，因此读操作数在周期 6。
- MUL.D F4, F2, F6 同样等 F2、F6，周期 6 读，乘法执行 10 周期，所以周期 16 完成，17 写回。
- SUB.D F8, F4, F2 写 F8，与前面 ADD.D F8, ... 有 WAW，必须等前一条写回后才能流出，所以较晚。
- ADD.D F4, F2, F6 写 F4，与 MUL.D F4, ... 有 WAW，也要等。

考试中不要只背表，要会解释“为什么这条卡住”：结构冲突、RAW、WAR、WAW 分别找。

例题 3：记分牌状态表字段

功能部件状态表常见字段：

字段 含义

--- ---

Busy 功能部件是否忙

Op 当前操作

Fi 目的寄存器

Fj, Fk 源寄存器

Qj, Qk 产生源操作数的功能部件

Rj, Rk 源操作数是否已经就绪

寄存器结果状态表：

Result[Fi] = 将要写 Fi 的功能部件名

若某寄存器没有等待写回的生产者，则为空。

例题 4：Tomasulo 算法怎么想

Tomasulo 核心思想：

1. 用保留站保存等待执行的指令。
2. 源操作数可用时记录值 Vj/Vk；不可用时记录生产者 tag Qj/Qk。
3. CDB 广播结果，等待同一 tag 的保留站同时捕获。
4. 寄存器结果状态保存 tag，从而实现寄存器重命名。

所以 Tomasulo：

RAW 真相关：不能消除，只能等值广播。□WAR/WAW 名相关：通过寄存器重命名消除。

作业中同一序列的 Tomasulo 调度结果为：

指令 流出 执行完成 写回

--- ---: ---: ---:

L.D F6, 34(R2) 1 2 3

L.D F2, 45(R3) 2 3 4

ADD.D F8, F6, F2 3 6 7

MUL.D F4, F2, F6 4 14 15

SUB.D F8, F4, F2 5 17 18

ADD.D F4, F2, F6 8 10 11

对比记分牌：Tomasulo 更快，主要因为重命名消除了 WAR/WAW，使后面的 SUB.D、ADD.D 可以更早流出。

8.4 作业题考前背法

1. 第一次作业：看到“CPI/MIPS/执行时间”，先列 $CPU\ time = IC \times CPI \times Cycle\ Time$ 。
2. 第一次作业：看到多个程序性能平均，问自己是“时间”“相对性能”还是“速率”。

3. 第二次作业：看到流水线时空图，先找瓶颈段和总完成时间。
4. 第二次作业：看到预约表，按同一行位置差求禁止表。
5. 第三次作业：看到记分牌，按“结构冲突、WAW、RAW、WAR”逐条卡。
6. 第三次作业：看到 Tomasulo，记住 Q 是等谁，V 是已有值，寄存器状态表存 tag。

9. 高频综合题模板

9.1 性能题模板

遇到性能题，按顺序写：

CPU time = IC × CPI × Cycle Time □ 若只比较同一程序且 IC 相同，比较 CPI × Cycle Time □ 若有局部优化，用 Amdahl □ 若有停顿，CPI = 理想 CPI + 各类停顿 CPI

9.2 流水线题模板

1. 写出阶段：IF、ID、EX、MEM、WB。
2. 标出每条指令读写寄存器。
3. 判断 RAW/WAR/WAW。
4. RAW 优先考虑旁路，不能旁路再停顿。
5. 分支题用 分支频率 × 失败率 × 惩罚。

9.3 Cache 题模板

1. 算块大小得到 offset。
2. 算行数或组数得到 index。
3. 剩余位数是 tag。
4. 访问序列题逐项模拟：先算块号，再算行号/组号，看 tag 是否匹配。
5. AMAT 题注意 miss rate 是百分数还是小数。

9.4 动态调度题模板

1. RAW：真相关，必须等待数据。
2. WAR/WAW：名相关，可用寄存器重命名消除。
3. Tomasulo：看保留站、tag、CDB。
4. ROB：看是否按序提交，是否支持精确异常。

10. 考前最后一页

必背公式：

CPU time = IC × CPI × Cycle Time □ CPI = $\sum (\text{比例}_i \times \text{CPI}_i)$ □ Speedup = $T_{\text{old}} / T_{\text{new}}$ □ Amdahl = $1 / ((1 - f) + f/S)$
 □ 流水时间 = $(k + n - 1) \times \Delta t$ □ 流水加速比 = $nk / (k + n - 1)$ □ AMAT = Hit Time + Miss Rate × Miss Penalty □ 多级 AMAT
 $T = HT_1 + MR_1 \times (HT_2 + MR_2 \times MP_2)$ □ Availability = $MTTF / (MTTF + MTTR)$ □ 串联系统失效率 = $\sum (1/MTTF_i)$

必背判断：

RAW = 写后读，真相关，不能靠重命名消除 □ WAR = 读后写，名相关，可重命名 □ WAW = 写后写，名相关，可重命名 □ 直接映像 = 位置唯一，简单快但冲突多 □ 全相联 = 任意位置，冲突少但硬件复杂 □ 组相联 = 折中 □ 写直达 = 同时写 Cache 和主存，简单但流量大 □ 写回 = 只写 Cache，替换时写回，复杂但流量小 □ 写分配 = 写不命中时先把块调入 Cache □ 不写分配 = 写不命中时直接写下一级

最后建议：先把第 1、3、5、7、8 章的计算题练熟，再回头背第 2 章 ISA 概念和第 4 章向量机概念。期末卷通常不是考“记住多少名词”，而是考你能不能把公式、冲突类型、Cache 地址划分这三类东西算稳。